

Домашнее задание №8

Дополнительное задание

1. Реализовать класс Автомобиль. У класса есть поля "Номер автомобиля"; Модель; Цвет; Пробег; Стоимость, свойства и методы.
2. Проверить работу в классе Main, методе main
3. Создать объект Java Collections со списком автомобилей.
4. Используя Java Stream API, вывести:
 - 1) Номера всех автомобилей, имеющих заданный в переменной цвет colorToFind или нулевой пробег mileageToFind.
 - 2) Количество уникальных автомобилей в ценовом диапазоне от n до m
Количество уникальных моделей в ценовом диапазоне от n до m.
 - 3) Вывести цвет автомобиля с минимальной стоимостью
 - 4) Среднюю стоимость искомой модели modelToFind
5. Реализовать ввод и вывод программы в файл *.txt
6. Вынести методы работы с автомобилем в папку repository интерфейс CarsRepository и его реализацию CarsRepositoryText
7. Доработать программу до следующей структуры
 - * файл с данными по пути \src\Homeworks\Homework08\data\cars.txt
 - * файл с результатами по пути \src\Homeworks\Homework08\data\result.txt
8. CarsRepositoryText обеспечивает уникальность номера автомобиля в списке

<https://github.com/Sel00k/JavaCourse>

Ветка Homework011 – основное задание

<https://github.com/Sel00k/JavaCourse/tree/Homework011>

Ветка Homework011Addition – дополнительное задание

<https://github.com/Sel00k/JavaCourse/tree/Homework011Addition>

cars.txt

car:a123me|Mercedes|White|0|8300000
car:b873of|Volga|Black|0|673000
car:w487mn|Lexus|Grey|76000|900000
car:p987hj|Volga|Red|610|704340
car:c987ss|Toyota|White|254000|761000
car:o983op|Toyota|Black|698000|740000
car:p146op|BMW|White|271000|850000
car:u893ii|Toyota|Purple|210900|440000
car:l097df|Toyota|Black|108000|780000
car:y876wd|Toyota|Black|160000|1000000
car:o983op|BMW|Red|133000|880000
colorToFind:Black
mileageToFind:0
PriceToFind:700000-800000
modelToFind:Toyota
modelToFind:Volvo
modelToFind:Volga

result.txt

Автомобиль с номером "o983op" уже есть

Автомобили в базе :

Номер	Модель	Цвет	Пробег	Цена
a123me	Mercedes	White	0	8300000
b873of	Volga	Black	0	673000
w487mn	Lexus	Grey	76000	900000
p987hj	Volga	Red	610	704340
c987ss	Toyota	White	254000	761000
o983op	Toyota	Black	698000	740000
p146op	BMW	White	271000	850000
u893ii	Toyota	Purple	210900	440000
l097df	Toyota	Black	108000	780000
y876wd	Toyota	Black	160000	1000000

Номера всех автомобилей, имеющих цвет "Black" или с 0 пробегом : [a123me, b873of, o983op, l097df, y876wd]

Количество уникальных автомобилей в ценовом диапазоне от 700000 до 800000 : 4

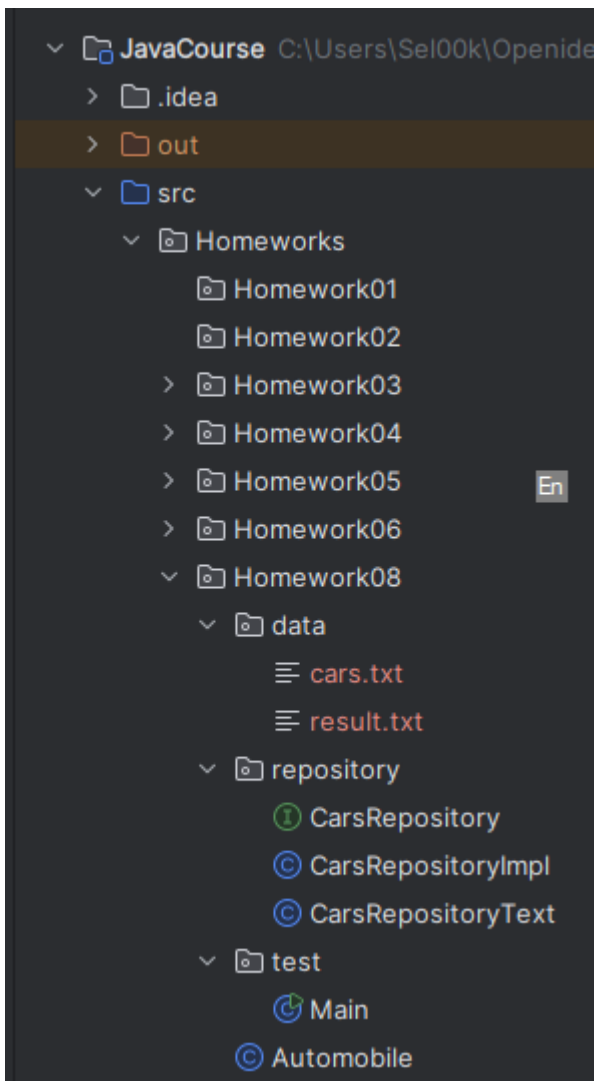
Количество уникальных моделей в ценовом диапазоне от 700000 до 800000 : 2

Цвет автомобиля с минимальной стоимостью : Purple

Среднюю стоимость модели "Toyota" : 744200,000000

Среднюю стоимость модели "Volvo" : 0,000000

Среднюю стоимость модели "Volga" : 688670,000000



CarsRepository.java

```
package Homeworks.Homework08.repository;

public interface CarsRepository {

    public void readCars();

    public void writeCars();

    public void writeNumbersFromColorAndMileag();

    public void writeUniqueCars();

    public void writeUniqueModels();

    public void writeColorMinPrice();

    public void writeAvePriceModel();
}
```

CarsRepositoryText.java

```
package Homeworks.Homework08.repository;

import Homeworks.Homework08.Automobile;

import java.io.*;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;

public class CarsRepositoryText implements CarsRepository{

    private List<Automobile> cars;
    private String colorToFind;
    private long mileageToFind;
    private List<String> modelsToFind = new ArrayList<>();
    private String fileNameCars;
    private String fileNameResult;
    private long priceStart;
    private long priceEnd;

    public CarsRepositoryText( String fileNameCars , String fileNameResult ) {
        this.cars = new ArrayList<>();
        this.fileNameCars = fileNameCars;
        this.fileNameResult = fileNameResult;
        readCars();
    }

    public boolean addCar( Automobile car ) {
        String number = car.getNumber();
        long c = this.cars.stream().filter( a -> a.getNumber().equalsIgnoreCase( number ) ).count();

        if ( c == 0 ) {
            this.cars.add( car );
            return true;
        }

        return false;
    }

    public boolean colorToFind( String colorToFind ) {
        this.colorToFind = colorToFind;
        return true;
    }

    public boolean priceToFind( long priceStart , long priceEnd ) {
        this.priceStart = priceStart;
        this.priceEnd = priceEnd;
        return true;
    }

    public boolean addModelToFind( String modelToFind ) {
        if ( !this.modelsToFind.contains( modelToFind ) ) {
            this.modelsToFind.add( modelToFind );
            return true;
        }

        return false;
    }
}
```

```

public boolean mileageToFind( long mileageToFind ) {
    this.mileageToFind = mileageToFind;
    return true;
}

public void readCars() {
    File fileCars    = new File( this.fileNameCars );
    File fileResult = new File( this.fileNameResult );
    String[] x , z;
    String var , value;
    Automobile car;
    List<String> ss = new ArrayList<>();

    if ( fileCars.exists() ) {
        try ( BufferedReader reader = new BufferedReader( new FileReader( fileCars ) ) ) {
            String line;

            while ( ( line = reader.readLine() ) != null ) {

                x = line.trim().split( ":" );

                if ( x.length == 2 ) {
                    var = x[ 0 ].trim().toLowerCase();
                    value = x[ 1 ].trim();

                    if ( var.equals( "car" ) ) {
                        z = value.split( "[|]" );
                        car = new Automobile( z[ 0 ] , z[ 1 ] , z[ 2 ] ,
                            Long.valueOf( z[ 3 ] ) , Long.valueOf( z[ 4 ] ) );

                        if ( !addCar( car ) ) {
                            ss.add( "Автомобиль с номером \" " + z[ 0 ] + "\" уже есть" );
                        }
                    }

                    if ( var.equals( "colortofind" ) ) {
                        colorToFind( value );
                    }

                    if ( var.equals( "mileagetofind" ) ) {
                        mileageToFind( Long.valueOf( value ) );
                    }

                    if ( var.equals( "pricetofind" ) ) {
                        z = value.split( "-" );
                        priceToFind( Long.valueOf( z[ 0 ] ) , Long.valueOf( z[ 1 ] ) );
                    }

                    if ( var.equals( "modeltofind" ) ) {
                        addModelToFind( value );
                    }
                }
            }
        } catch ( IOException e ) {
            e.printStackTrace(); // Обработка возможных ошибок ввода-вывода
        }
    }
}

```

```

if ( fileResult.exists() ) {
    try ( BufferedWriter writer = new BufferedWriter( new FileWriter(fileResult,true) ) ) {
        for ( String s : ss ) {
            writer.write( s ); writer.newLine();
        }

        writer.newLine();
    } catch ( IOException e ) {
        e.printStackTrace();
        System.err.println( "Ошибка при записи в файл." );
    }
}

}

public void writeCars() {
    File fileResult = new File( this.fileNameResult );
    String t1 = "+-----+-----+-----+-----+";
    String t2 = "| Номер | Модель | Цвет | Пробег | Цена |";
    String t3;

    try ( BufferedWriter writer = new BufferedWriter( new FileWriter( fileResult , true ) ) ) {
        writer.write( "Автомобили в базе :" ); writer.newLine();
        writer.write( t1 ); writer.newLine();
        writer.write( t2 ); writer.newLine();
        writer.write( t1 ); writer.newLine();

        for ( Automobile car : cars ) {
            t3 = car.toString();
            writer.write( t3 ); writer.newLine();
        }

        writer.write( t1 ); writer.newLine();
        writer.newLine();
    } catch ( IOException e ) {
        e.printStackTrace();
        System.err.println( "Ошибка при записи в файл." );
    }
}

public void writeNumbersFromColorAndMileag() {
    File fileResult = new File( this.fileNameResult );
    List<String> r1 = getNumbersFromColorAndMileage( colorToFind , mileageToFind );
    String s = String.format(
        "Номера всех автомобилей, имеющих цвет \"%s\" или с %d пробегом : %s" ,
        colorToFind , mileageToFind , r1 );

    try ( BufferedWriter writer = new BufferedWriter( new FileWriter( fileResult , true ) ) ) {
        writer.write( s ); writer.newLine();
    } catch ( IOException e ) {
        e.printStackTrace();
        System.err.println( "Ошибка при записи в файл." );
    }
}

public void writeUniqueCars() {
    File fileResult = new File( this.fileNameResult );
    long r2 = countUniqueCars( priceStart , priceEnd );
    String s = String.format(
        "Количество уникальных автомобилей в ценовом диапазоне от %d до %d : %d" ,
        priceStart , priceEnd , r2 );

```

```

    try ( BufferedWriter writer = new BufferedWriter( new FileWriter( fileResult , true ) ) ) {
        writer.write( s ); writer.newLine();
    } catch ( IOException e ) {
        e.printStackTrace();
        System.err.println( "Ошибка при записи в файл." );
    }
}

public void writeUniqueModels() {
    File fileResult = new File( this.fileNameResult );
    long r2 = countUniqueModels( priceStart , priceEnd );
    String s = String.format(
        "Количество уникальных моделей в ценовом диапазоне от %d до %d : %d" ,
        priceStart , priceEnd , r2 );

    try ( BufferedWriter writer = new BufferedWriter( new FileWriter( fileResult , true ) ) ) {
        writer.write( s ); writer.newLine();
    } catch ( IOException e ) {
        e.printStackTrace();
        System.err.println( "Ошибка при записи в файл." );
    }
}

public void writeColorMinPrice() {
    File fileResult = new File( this.fileNameResult );
    String r3 = getColorMinPrice();
    String s = String.format( "Цвет автомобиля с минимальной стоимостью : %s" , r3 );

    try ( BufferedWriter writer = new BufferedWriter( new FileWriter( fileResult , true ) ) ) {
        writer.write( s ); writer.newLine();
    } catch ( IOException e ) {
        e.printStackTrace();
        System.err.println( "Ошибка при записи в файл." );
    }
}

public void writeAvePriceModel() {
    File fileResult = new File( this.fileNameResult );
    double r4;
    String s;

    try ( BufferedWriter writer = new BufferedWriter( new FileWriter( fileResult , true ) ) ) {
        for ( String modelToFind : modelsToFind ) {
            r4 = getAvePriceModel( modelToFind );
            s = String.format( "Среднюю стоимость модели \"%s\" : %f" , modelToFind , r4 );
            writer.write( s ); writer.newLine();
        }
    } catch ( IOException e ) {
        e.printStackTrace();
        System.err.println( "Ошибка при записи в файл." );
    }
}

// Номера всех автомобилей, имеющих заданный в переменной
// цвет colorToFind или нулевой пробег mileageToFind
public List<String> getNumbersFromColorAndMileage( String colorFind , long mileageFind ) {
    List<String> r1 = cars.stream()
        .filter( car -> car.getMileage() == mileageFind ||
            car.getColor().equalsIgnoreCase( colorFind ) )
        .map( Automobile::getNumber )
        .collect( Collectors.toList() );
    return r1;
}

```

```

// Количество уникальных автомобилей в ценовом диапазоне от n до m
public long countUniqueCars( long pStart , long pEnd ) {
    long r2a = cars.stream()
        .filter( car -> car.getPrice() >= pStart )
        .filter( car -> car.getPrice() <= pEnd )
        .map( Automobile::getNumber )
        .distinct()
        .count();
    return r2a;
}

// Количество уникальных моделей в ценовом диапазоне от n до m
public long countUniqueModels( long pStart , long pEnd ) {
    long r2b = cars.stream()
        .filter( car -> car.getPrice() >= pStart )
        .filter( car -> car.getPrice() <= pEnd )
        .map( Automobile::getModel )
        .distinct()
        .count();
    return r2b;
}

// цвет автомобиля с минимальной стоимостью
public String getColorMinPrice() {
    String r3 = "Null";
    Optional<Automobile> r3optional = cars.stream()
        .min( Comparator.comparingLong( Automobile::getPrice ) );
    if( r3optional.isPresent() ) {
        r3 = r3optional.get().getColor();
    }
    return r3;
}

// Среднюю стоимость искомой модели modelToFind
public double getAvePriceModel( String modelFind ) {
    double r4 = cars.stream()
        .filter( car -> car.getModel().equalsIgnoreCase( modelFind ) )
        .mapToLong( Automobile::getPrice )
        .average()
        .orElse( 0.0 );
    return r4;
}
}

```

Automobile.java

```
package Homeworks.Homework08;

import java.util.Objects;

public class Automobile {
    private String number;
    private String model;
    private String color;
    private long mileage;
    private long price;

    public Automobile( String number ) {
        this.number = number;
    }

    public Automobile( String number , String model , String color , long mileage , long price ) {
        this.number = number;
        this.model = model;
        this.color = color;
        this.mileage = mileage;
        this.price = price;
    }

    public String getNumber() {
        return number;
    }

    public void setNumber( String number ) {
        this.number = number;
    }

    public String getModel() {
        return model;
    }

    public void setModel( String model ) {
        this.model = model;
    }

    public String getColor() {
        return color;
    }

    public void setColor( String color ) {
        this.color = color;
    }

    public long getMileage() {
        return mileage;
    }

    public void setMileage( long mileage ) {
        this.mileage = mileage;
    }

    public long getPrice() {
        return price;
    }
}
```

```

public void setPrice( long price ) {
    this.price = price;
}

@Override
public boolean equals(Object o) {
    if (o == null || getClass() != o.getClass()) return false;
    Automobile that = (Automobile) o;
    return mileage == that.mileage && price == that.price
        && Objects.equals(number, that.number)
        && Objects.equals(model, that.model)
        && Objects.equals(color, that.color);
}

@Override
public int hashCode() {
    return Objects.hash(number, model, color, mileage, price);
}

@Override
public String toString() {
    return String.format( "| %-6s | %-8s | %-6s | %6d | %7d | " ,
        number , model , color , mileage , price );
}
}

```

Main.java

```
package Homeworks.Homework08.test;

/* === Домашнее задание №8 по теме «Java Collections. Stream API», дополнительное задание ===
1. Реализовать класс Автомобиль. У класса есть поля "Номер автомобиля"; Модель; Цвет; Пробег; Стоимость,
   свойства и методы.
2. Проверить работу в классе Main, методе main.
3. Создать объект Java Collections со списком автомобилей.
4. Используя Java Stream API, вывести:
   1) Номера всех автомобилей, имеющих заданный в переменной
      цвет colorToFind или нулевой пробег mileageToFind.
   2а) Количество уникальных автомобилей в ценовом диапазоне от n до m
   2б) Количество уникальных моделей в ценовом диапазоне от n до m.
   3) Вывести цвет автомобиля с минимальной стоимостью.
   4а) Среднюю стоимость искомой модели modelToFind
   4б) Среднюю стоимость искомой модели modelToFind1
   4в) Среднюю стоимость искомой модели modelToFind2
5. Реализовать ввод и вывод программы в файл *.txt.
6. Вынести методы работы с автомобилем в папку repository интерфейс
   CarsRepository и его реализацию CarsRepositoryText.
7. Доработать программу до следующей структуры
   * файл с данными по пути \src\Homeworks\Homework08\data\cars.txt
   * файл с результатами по пути \src\Homeworks\Homework08\data\result.txt
8. CarsRepositoryText обеспечивает уникальность номера автомобиля в списке
*/

import Homeworks.Homework08.repository.CarsRepositoryText;

public class Main {

    public static void main( String[] args )
    {
        String currentDirectory = System.getProperty( "user.dir" );
        System.out.println( "Текущая рабочая директория: " + currentDirectory );
        String fileNameCars    = currentDirectory + "\\src\\Homeworks\\Homework08\\data\\cars.txt";
        String fileNameResult  = currentDirectory + "\\src\\Homeworks\\Homework08\\data\\result.txt";

        CarsRepositoryText carsRepository = new CarsRepositoryText( fileNameCars , fileNameResult );

        carsRepository.writeCars();
        carsRepository.writeNumbersFromColorAndMileag();
        carsRepository.writeUniqueCars();
        carsRepository.writeUniqueModels();
        carsRepository.writeColorMinPrice();
        carsRepository.writeAvePriceModel();
    }
}
```